



UNIVERSIDAD DE CARABOBO
FACULTAD EXPERIMENTAL DE CIENCIAS
Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN
ARQUITECTURA DEL COMPUTADOR



PRÁCTICA DE LABORATORIO #4

Gestión de Entrada/Salida, Interrupciones y Rutinas de Servicio en Arquitectura MIPS32

Estudiantes:

Nathaly Campos *C.I. 32.460.314*

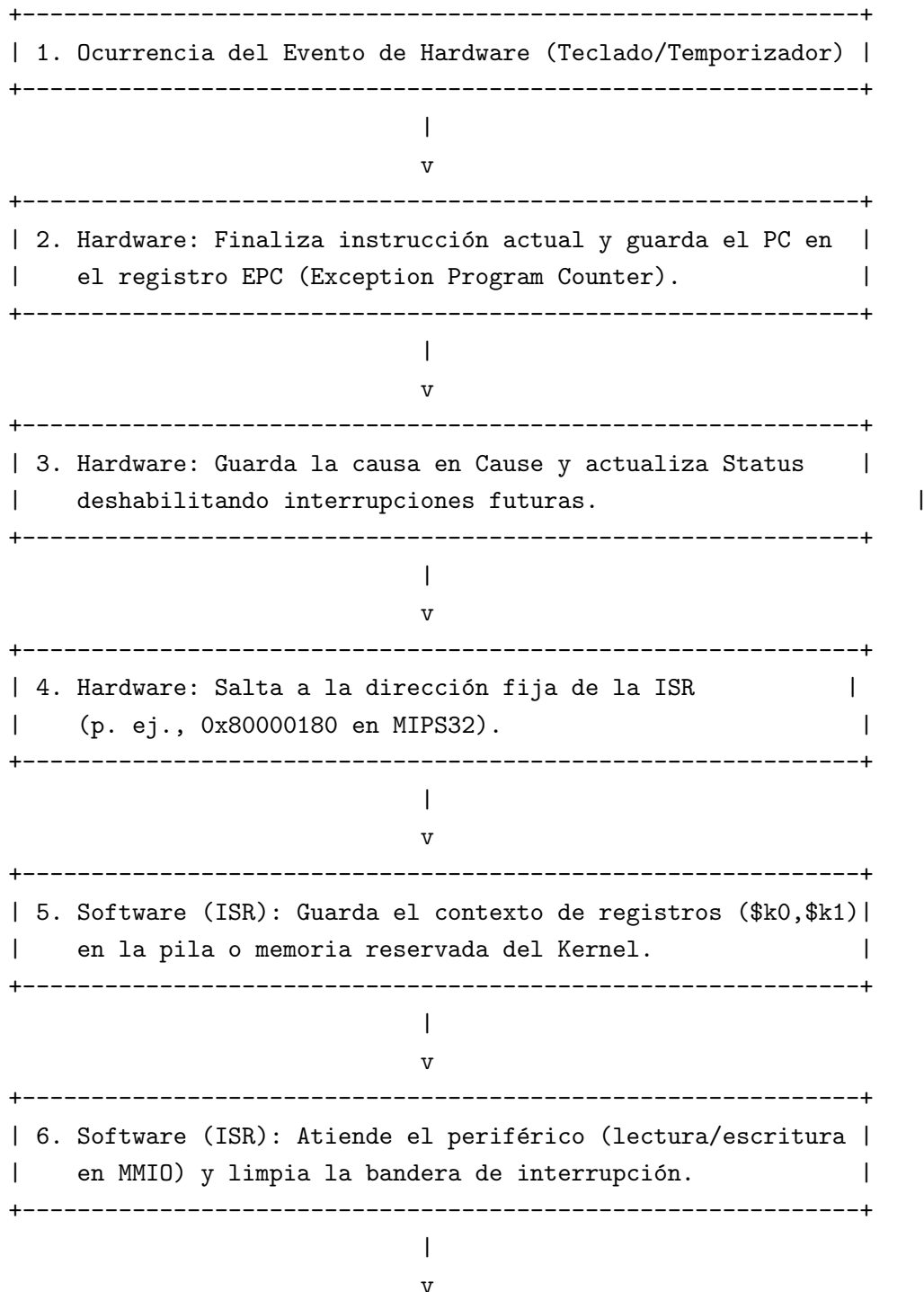
David Serrano *C.I. 31.000.672*

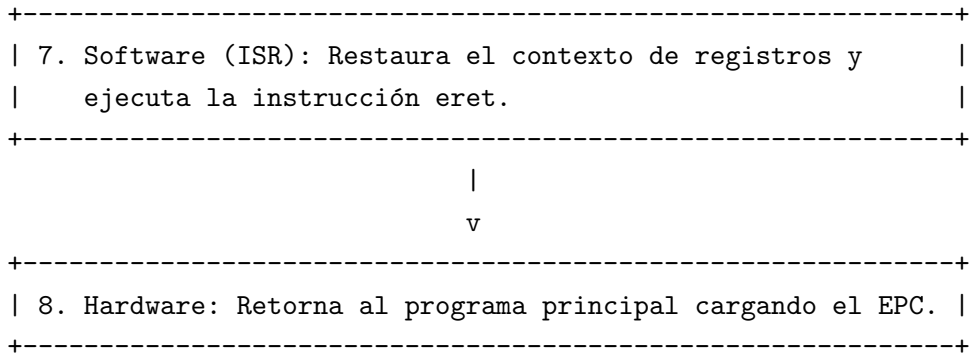
Profesor:

José Canache

1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).

El ciclo de atención a una interrupción de hardware sigue una secuencia coordinada entre el hardware del procesador y el software (sistema operativo / rutina de servicio):





2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

- **Gestión con Sondeo (Polling):** El procesador consulta de forma continua y activa el registro de estado del dispositivo de E/S (como la dirección MMIO 0xffff0000) mediante un bucle cerrado. La CPU no puede realizar otras tareas útiles mientras espera un evento, consumiendo ciclos de reloj innecesarios.
- **Gestión con Interrupciones:** El procesador continúa ejecutando el programa principal libremente. Cuando el dispositivo periférico requiere atención (por ejemplo, al presionar una tecla), envía una señal eléctrica de interrupción a la CPU. La CPU detiene la ejecución actual, atiende el periférico a través de la rutina de servicio (ISR) y reanuda inmediatamente su labor anterior.

3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

- **Eficiencia en el uso de CPU:** Evita el desperdicio de millones de ciclos de procesamiento consumidos en bucles de espera activa.
- **Soporte para Multitarea y Tiempo Real:** Permite que el sistema procese múltiples eventos concurrentes (temporizadores, teclados, redes) garantizando latencias de respuesta mínimas.
- **Ahorro Energético:** La CPU puede entrar en modos de bajo consumo o ejecutar hilos de fondo mientras no existan eventos pendientes de periféricos.

4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

La gestión se realiza mediante los registros del **Coprocesador 0 (CP0)**:

- **EPC (Exception Program Counter - Registro 14):** Almacena la dirección de la instrucción que fue interrumpida o que generó la excepción para retornar a ella.
- **Cause (Registro 13):** Contiene un código de causa (bits 2 al 6) que indica la razón del evento (interrupción, desbordamiento, fallo de dirección) y los bits de interrupción pendientes (IP).
- **Status (Registro 12):** Registro de control de estado. Contiene las máscaras para habilitar/deshabilitar interrupciones a nivel global (IE) y por dispositivo (IM).
- **BadVAddr (Registro 8):** Registra la dirección de memoria virtual que intentó un acceso no válido cuando ocurre un error de dirección.

5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

Puesto que las interrupciones son eventos **asíncronos** e impredecibles, interrumpe el programa principal en cualquier punto de su ejecución. Si la Rutina de Servicio de Interrupción (ISR) modifica los registros del banco general (\$t0, \$s0, \$a0, etc.) sin preservarlos, alterará las variables y estados temporales del programa interrumpido. Guardar el contexto asegura la **transparencia de la interrupción**, de modo que al restaurar los registros y ejecutar `eret`, el programa interrumpido continúe exactamente en el mismo estado sin corrupción de datos.

6. Momentos en que pueden generarse excepciones en un sistema MIPS32.

a) Situaciones comunes de excepción:

- *Desbordamiento Aritmético (Arithmetic Overflow):* Ocurre con instrucciones de suma/resta con signo (`add`, `sub`).
- *Fallo de Dirección (Address Error):* Lectura o escritura de palabra no alineada a múltiplo de 4 bytes (`lw/sw`).
- *Instrucción Reservada (Reserved Instruction):* Intento de ejecutar un código de operación no válido o no soportado.
- *Llamada al Sistema / Trap (syscall):* Excepción provocada voluntariamente por software para solicitar servicios del SO.

b) Excepciones en las etapas del pipeline:

- **IF (Fetch):** Error de alineación en la búsqueda de instrucciones o fallo en el caché de instrucciones.
- **ID (Decode):** Detección de instrucción no válida o violación de privilegios.
- **EX (Execute):** Excepciones aritméticas (desbordamiento) o trampas condicionales.
- **MEM (Memory):** Fallos de alineación de direcciones de datos o violaciones de protección de memoria RAM.

7. Estrategias de tratamiento de excepciones e interrupciones

a) Diferencia entre Interrupciones y Excepciones:

- *Interrupciones (Asíncronas):* Generadas por fuentes externas al procesador (teclado, temporizador, red). Ocurren independientemente de la instrucción en ejecución.
- *Excepciones (Síncronas):* Generadas internamente por la ejecución de una instrucción específica dentro de la CPU (división por cero, desbordamiento, instrucción ilegal).

b) Estrategias y Función del EPC:

- *Redirección de Ejecución:* MIPS32 utiliza un vector de excepciones o una dirección fija de inicio (p. ej., 0x80000180). Al detectarse un evento, el hardware fuerza la carga de esta dirección en el PC.
- *Función del Registro EPC:* Almacena la dirección exacta de la instrucción interrumpida (o PC - 4 en el caso de estar en una ranura de retardo *branch delay slot*). Es indispensable para restaurar la ejecución mediante la instrucción especial **eret**.

8. Habilitación de interrupciones en dispositivos y procesador

a) Pasos de Habilitación:

- *En el Teclado:* Se establece en 1 el bit 1 (*Interrupt Enable*) del Receiver Control Register en la dirección MMIO 0xffff0000.
- *En la Pantalla:* Se habilita asignando 1 al bit 1 del Transmitter Control Register (0xffff0008).
- *En el Procesador:* Se deben modificar los bits del registro **Status** del CP0: el

bit 0 (**IE** - *Interrupt Enable*) debe colocarse en 1, y los bits de máscara de interrupción (**IM**, bits 8 al 15) deben habilitar la línea específica correspondiente al periférico.

- b) **Efecto de Habilidad Parcial:** Si se habilitan las interrupciones en los dispositivos periféricos pero el bit global **IE** del registro **Status** permanece en 0, las interrupciones estarán bloqueadas a nivel de la CPU. El hardware ignorará las peticiones externas, obligando al sistema a funcionar bajo un esquema de sondeo explícito o perdiendo las peticiones.

9. Procesamiento de interrupciones

a) **Paso a paso de una interrupción de reloj:**

1. El temporizador cumple la cuenta e impulsa la línea de interrupción de hardware.
2. El Hardware guarda el PC actual en el registro **EPC**, actualiza el registro **Cause** y conmuta el bit **IE** a 0 en **Status**.
3. La CPU salta a la rutina de manejo de interrupciones (0x80000180).
4. La rutina (Software) guarda los registros de trabajo del kernel (**\$k0**, **\$k1**) y los registros del contexto de usuario necesarios en la pila.
5. La rutina limpia el flag de interrupción en el hardware del reloj y ejecuta la tarea de control de tiempo.
6. La rutina restaura los registros generales desde la pila.
7. La instrucción **eret** restaura el registro **Status** y carga el valor de **EPC** en el PC.

- b) **Importancia de guardar el contexto:** Al estar involucrados procesos de usuario y eventos del kernel, no preservar los registros generales, el **EPC** o el **Status** causaría colisiones irrecuperables cuando ocurran interrupciones anidadas o retornos a programas interrumpidos.

10. Interrupciones de reloj y control de ejecución

- a) **Control de bucles infinitos y tiempos límite:** Las interrupciones de reloj actúan como un guardián del sistema (*Watchdog Timer*). Al configurarse para activarse periódicamente, la ISR decrementa un contador global de tiempo asignado al programa. Si el contador llega a cero antes de terminar, el sistema operativo aborta el programa atascado en el bucle.
- b) **Finalización anticipada por software:** Si el programa finaliza su ejecución antes de agotarse el tiempo del reloj, debe invocar explícitamente una llamada al sistema (`syscall` de salida / termina) que deshabilite la alarma programada en el temporizador para evitar interrupciones espurias.

11. Análisis y Discusión de los Resultados

- **Ejercicio 1 (Buffer Circular):** La prueba realizada en la herramienta *Keyboard and Display MMIO Simulator* validó la captura de teclas mediante la dirección `0xffff0000`. El filtrado ASCII en el rango `[65, 90]` descartó efectivamente símbolos y minúsculas, acumulando solo letras mayúsculas en el arreglo circular durante la ventana de 20 segundos determinada por la llamada al sistema de tiempo.
- **Ejercicio 2 (Semáforo Interactante):** Se verificó la lógica de estados en tiempo real. La lectura de la tecla 's' desencadenó la respuesta inmediata del semáforo. La implementación de la rutina de retardo garantizó la suspensión precisa de la ejecución en lapsos de 20s, 10s y 30s respectivamente, asegurando un ciclo continuo sin bloquear la capacidad del simulador.